Aprendizado de Máquina





Classroom code tg44fjf

Prof. Rodrigo Caropreso
Nov/2018
rcaropreso at gmail

Conteúdo Programático – Aulas 1 e 2

Introdução

- Conceitos Gerais em Aprendizado de Máquina
- Tipos de Aprendizado: Supervisionado, Semi-Supervisionado, Não-Supervisionado;
- Aprendizado por Reforço.

Computação Evolutiva e Inteligência Coletiva

- Algoritmos Genéticos;
- Otimização por Enxame de Partículas (PSO).

Redes Neurais Artificiais – Parte 1

- Algoritmo de Descida de Gradiente
- Conceitos em Redes Neurais Artificiais (RNA);
- Perceptron Simples, Adaline;
- Lei de Aprendizado de Hebb;
- Regra Delta.

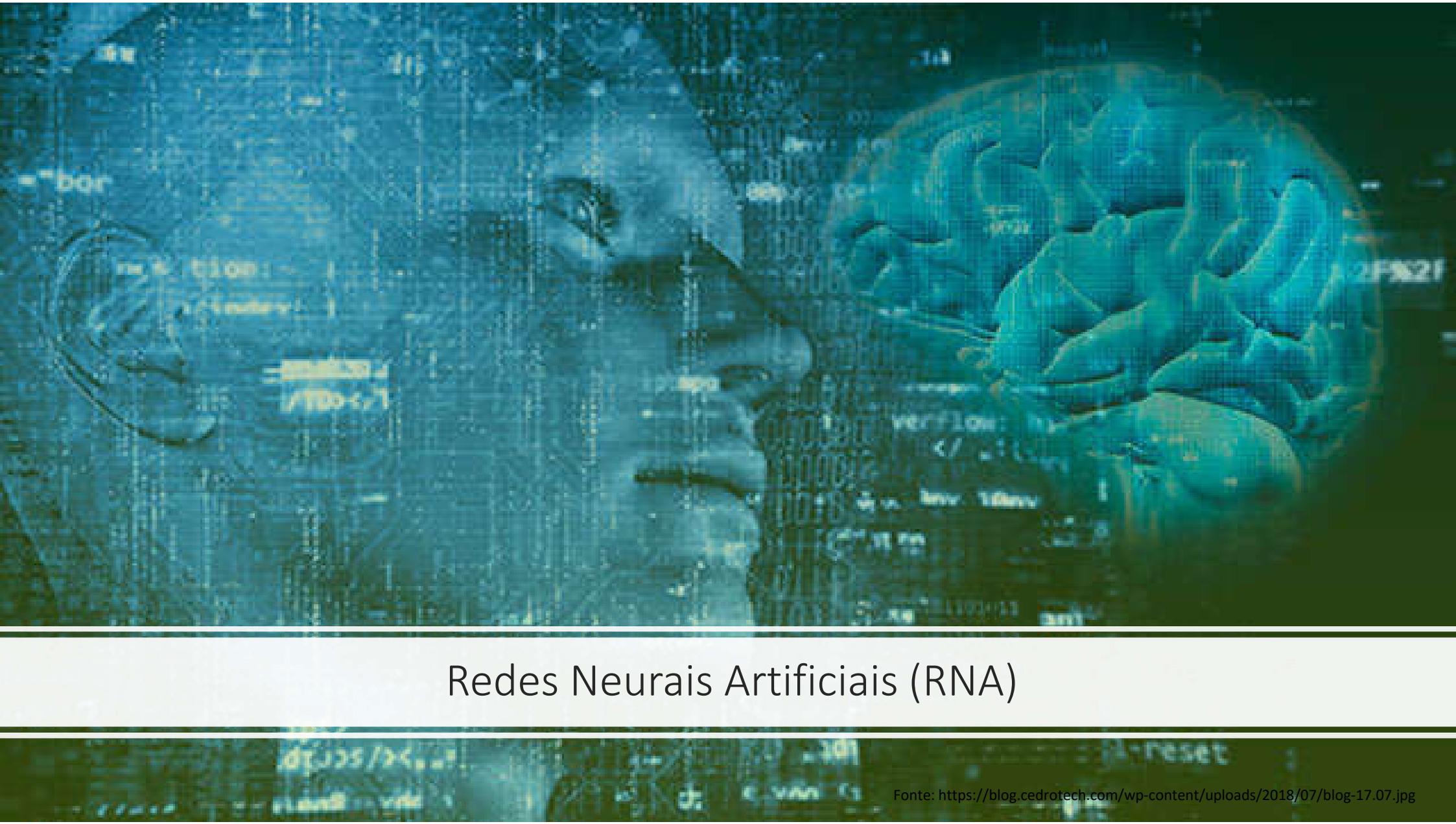
Conteúdo Programático – Aulas 3 e 4

Redes Neurais Artificiais – Parte 2

- Clusterização e K-Means;
- Classificação e K-NN;
- Redes de Base Radial;

Análise Estatística

- Problemas de Classificação e Regressão;
- Regressão Linear e Logística;
- Parâmetros de Regularização;
- Naive Bayes.



Redes Neurais Artificiais (RNA)

Fonte: https://blog.cedrotech.com/wp-content/uploads/2018/07/blog-17.07.jpg

Redes Neurais Artificiais (RNA): Conceitos



- São modelos computacionais inspirados nos mecanismos de aprendizagem do cérebro humano.
- São modelos computacionais que tentam emular a forma com que o cérebro resolve problemas.

“Uma Rede Neural Artificial é uma estrutura que processa informação de forma paralela e distribuída e que consiste de unidades computacionais interconectadas. O processamento executado em cada unidade pode ser definido arbitrariamente e deve depender dos sinais de entrada que chegam até a unidade através de suas conexões.”

(NASCIMENTO e YONEYAMA, 2010)

Redes Neurais Artificiais (RNA): Conceitos



- O processamento de informações no cérebro humano é altamente complexo, não-linear e paralelo.
  - O cérebro é constituído por aproximadamente 100 bilhões de neurônios (10^{11}).
  - Cada neurônio é interligado em média com outros 6.000 neurônios (60 trilhões de sinapses).
  - Número de neurônios equivale à quantidade de estrelas da via Láctea.
  - É um dos sistemas mais complexos que é atualmente conhecido pelo ser humano.

Redes Neurais Artificiais (RNA): Conceitos



	Computador	**Cérebro**
Tipo de Processamento	sequencial	paralelo
Quantidade e Complexidade	Um ou poucos processadores	10^{10} neurônios; 10^{3} conexões/neurônio
Velocidade de Processamento	10^{-9} segundos	10^{-6} segundos
Eficiência Energética	10^{-16} joules por op./segundo	10^{-6} joules por op./segundo

Redes Neurais Artificiais (RNA): Propriedades



- Adaptação por Experiência
  - Parâmetros internos da rede são ajustados a partir da apresentação sucessiva de exemplos (amostras e medidas).
- Capacidade de Aprendizado
  - Aplicação de método de treinamento possibilita a rede extrair o
  - relacionamento existente entre variáveis que compõem a aplicação.
- Habilidade de Generalização
  - Após o processo de treinamento, a rede é capaz de generalizar o conhecimento adquirido, possibilitando a estimação de soluções que eram até então desconhecidas
- Organização de Dados
  - Baseada em características marcantes de um conjunto de dados a respeito do processo, a rede é capaz de realizar sua organização interna para agrupamento de amostras que são similares/comuns.

Redes Neurais Artificiais (RNA): Propriedades



- Tolerância a Falhas
  - Devido ao elevado nível de interconexões entre neurônios artificiais, a rede torna-se um sistema tolerante a falhas quando parte de sua estrutura interna for sensivelmente corrompida.
- Armazenamento Distribuído
  - O conhecimento dentro da rede é realizado de forma distribuída entre as sinapses dos neurônios artificiais, permitindo-se então robustez frente a eventuais neurônios que se tornaram inoperantes.
- Facilidade de Prototipagem
  - A implementação da maioria das arquiteturas neurais pode ser facilmente prototipadas em hardware ou em software.
  - Após o processo de treinamento, os seus resultados são normalmente obtidos por algumas operações matemáticas elementares.

Redes Neurais Artificiais (RNA): Aplicações



- Aproximação de Funções
  - Mapeiam o relacionamento entre variáveis de um sistema a partir de um conjunto conhecido de seus valores representativos.
  - Envolvem normalmente o mapeamento de processos cuja modelagem por técnicas convencionais são de difícil obtenção.

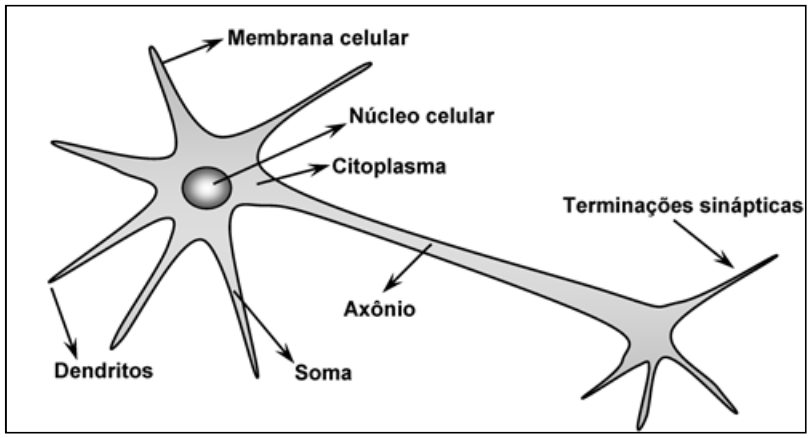
Redes Neurais Artificiais (RNA): Aplicações



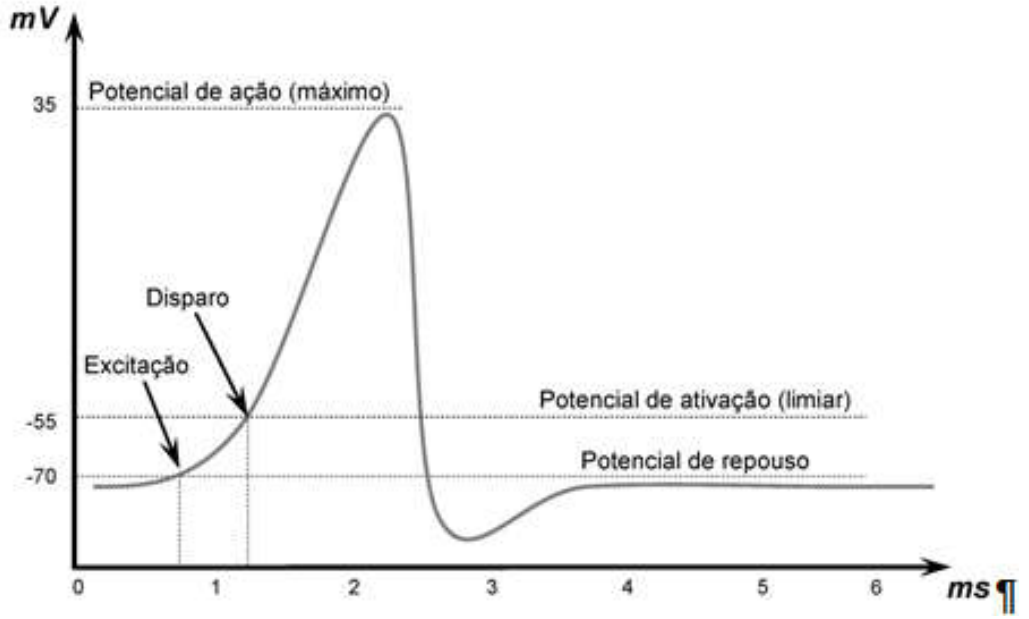
- Classificação de Padrões
  - O objetivo desta aplicação consiste de associar um padrão de entrada (amostra) para uma das classes previamente definidas.
  - O problema a ser tratado possui um conjunto discreto e conhecido das possíveis saídas desejadas.

Redes Neurais Artificiais (RNA): Modelo

Fluxo de Informações

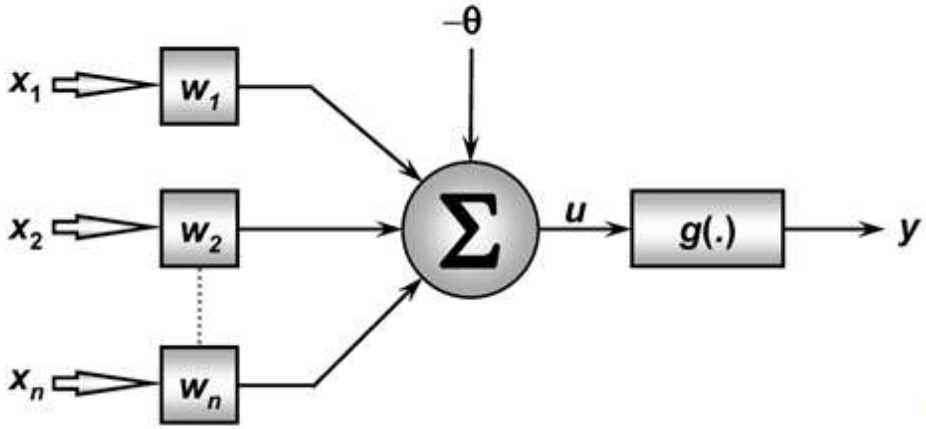


- Neurônio Biológico



Redes Neurais Artificiais (RNA): Neurônio Artificial





- **Entrada**: x_1, x_2,..., x_n
- **Pesos sinápticos**: w_1, w_2,..., w_n
- **Limiar de Ativação**: θ
- **Função de Ativação**: g(.)
- **Saída**: y

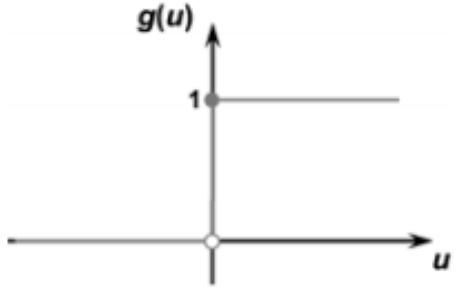
$$u = \sum_{i=1}^{n} w_i \cdot x_i - \theta$$

$$y = g(u)$$

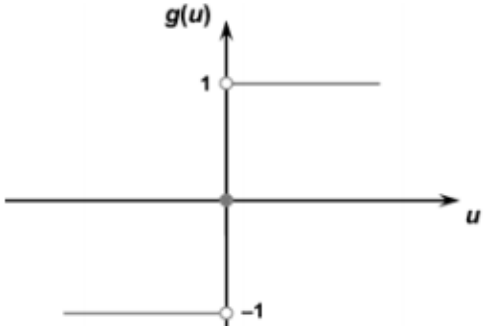
Redes Neurais Artificiais (RNA): Funções de Ativação



- Função Degrau $g(u) = \begin{cases} -1, u < 0 \\ 1, u \geq 0 \end{cases}$



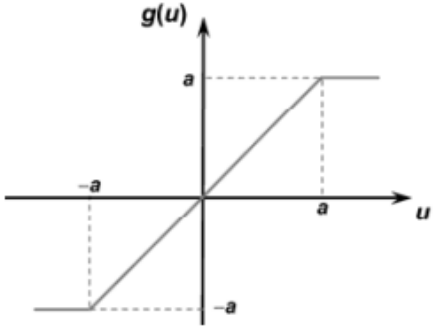
- Função Degrau Bipolar $g(u) = \begin{cases} 0, u < 0 \\ 1, u \geq 0 \end{cases}$



Redes Neurais Artificiais (RNA): Funções de Ativação

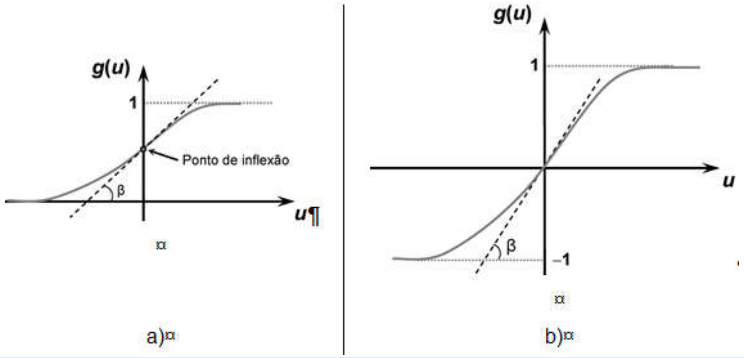


- Função Rampa



$$g(u) = \begin{cases} a, se\ u > a \\ u, se\ -a < u < a \\ -a, se\ u < -a \end{cases}$$

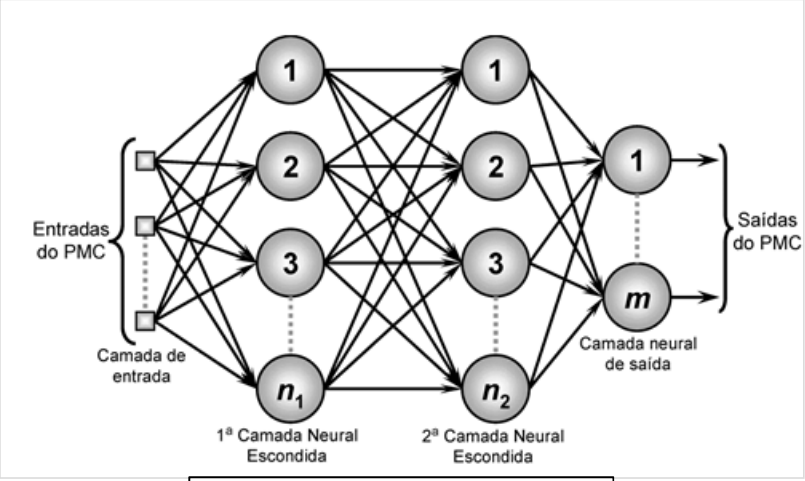
- Função Logística/Tangente Hipoerbólica



$$g(u) = \frac{1}{1 + e^{-\beta u}}$$

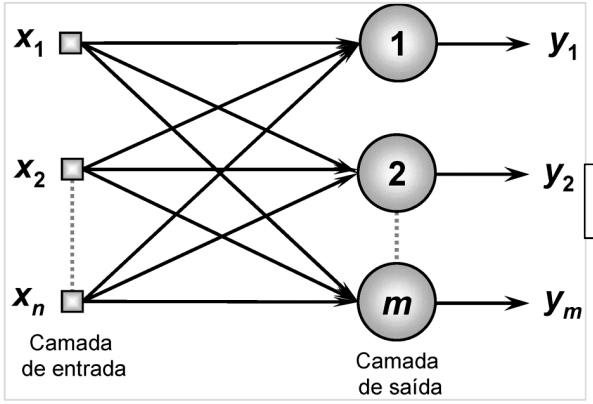
$$g(u) = \frac{1 - e^{-\beta u}}{1 + e^{-\beta u}}$$

Redes Neurais Artificiais (RNA): Arquitetura, Topologia e Treinamento



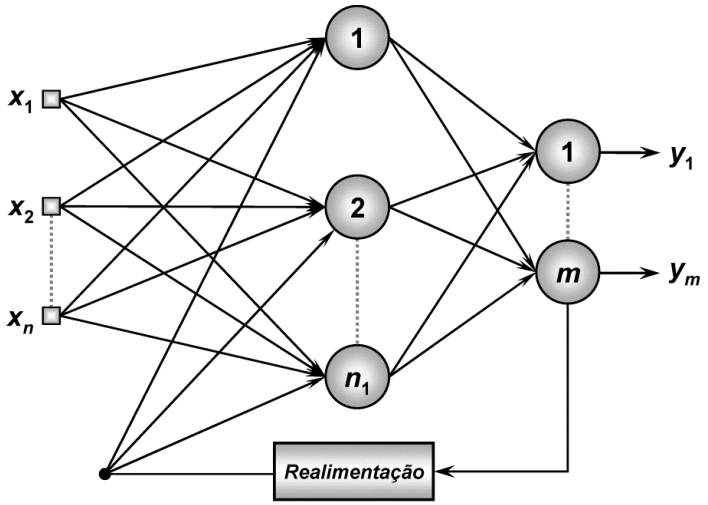
Multicamadas

- Topologia de Redes Neurais:
  - Define as diferentes formas de composições estruturais que uma rede poderá assumir, frente a uma determinada arquitetura. Exemplo: dada uma determinada arquitetura, uma das redes pode ser composta de 10 neurônios e a outra de 20 neurônios.



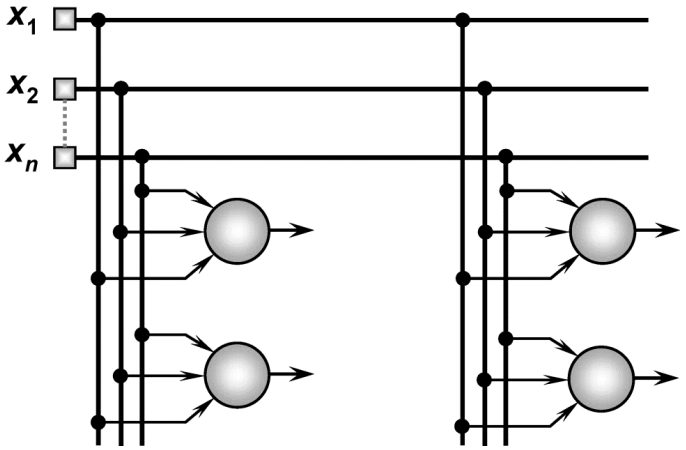
Camada Simples

Redes Neurais Artificiais (RNA): Arquitetura, Topologia e Treinamento



Recorrente

- Arquitetura de Redes Neurais:
  - Define a forma como os seus diversos neurônios estão arranjados, ou dispostos, uns em relação aos outros. Os arranjos são essencialmente estruturados através do direcionamento do fluxo sináptico.



Reticulada

Redes Neurais Artificiais (RNA): Arquitetura, Topologia e Treinamento

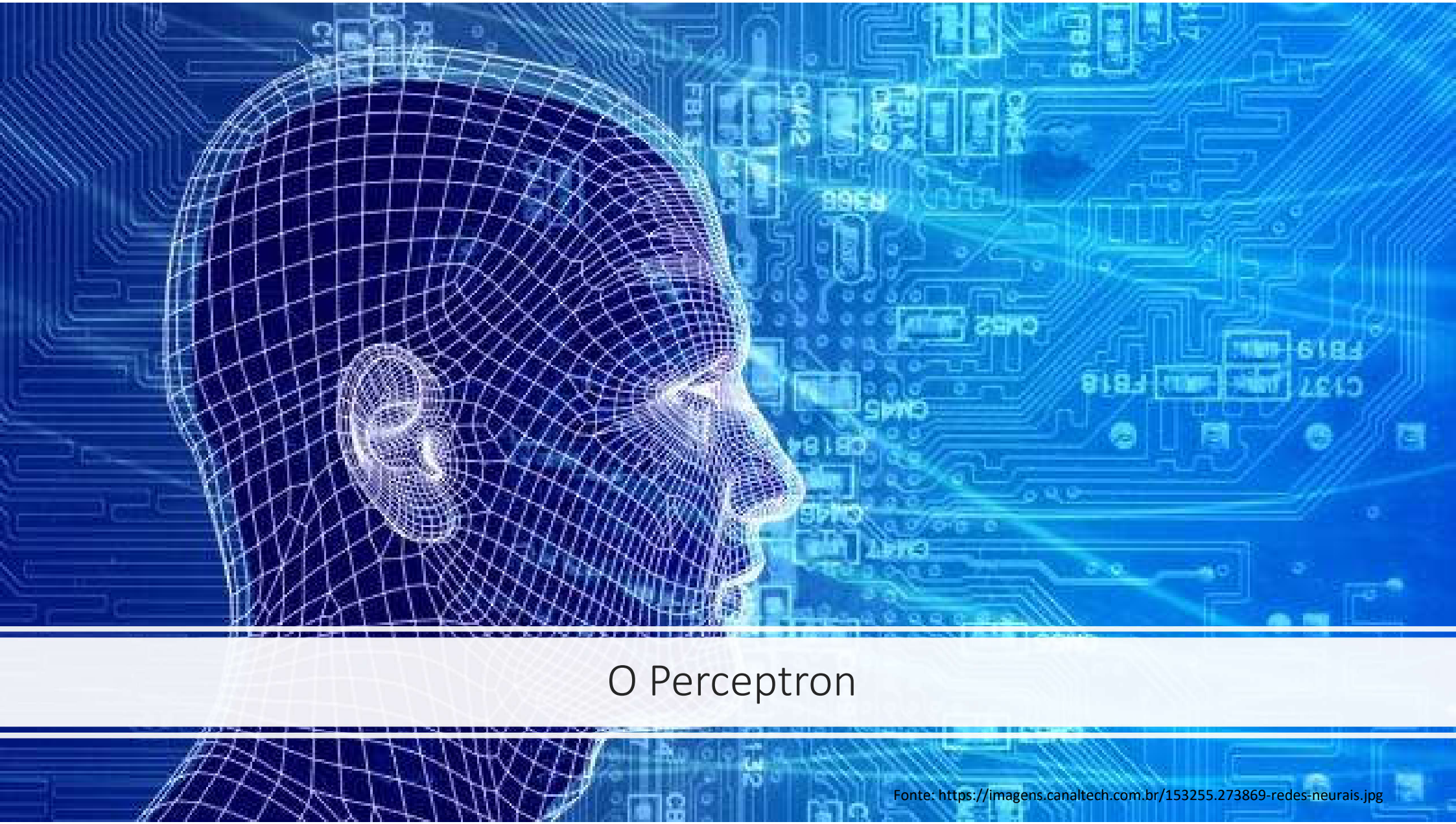


- Processo de Treinamento:
  - Consiste da aplicação de passos ordenados a fim de ajustar (sintonizar) os parâmetros livres (pesos sinápticos e limiares) dos neurônios.
- Conjuntos de Treinamento e Teste:
  - **Conjunto Total de Amostras**: Representa todos os dados disponíveis sobre o comportamento do processo a ser mapeado. Será dividido em subconjuntos de treinamento e de teste.
  - **Subconjunto de Treinamento**: Usado essencialmente para o processo de aprendizado da rede. Composto aleatoriamente com cerca de 60 a 90% das amostras do conjunto total.
  - **Subconjunto de Teste**: Usado p/ verificar se a generalização de soluções por parte da rede já estão em patamares aceitáveis. Composto de 10 a 40% das amostras do conjunto total.

Redes Neurais Artificiais (RNA): Arquitetura, Topologia e Treinamento



- Treinamento Supervisionado:
  - Consiste em se ter disponível, considerando cada amostra dos sinais de entrada, as respectivas saídas desejadas.
  - Comporta como se houvesse um “professor” ensinando para a rede qual seria a resposta correta para cada amostra apresentada.
- Passos do Treinamento Supervisionado:
  1. Apresente uma amostra de treinamento.
  2. Calcule a saída produzida pela rede.
  3. Compare com a saída desejada.
  4. Se estiver dentro de valores aceitáveis:
  5. Então Termine o processo de aprendizado.
  6. Senão Ajuste os pesos sinápticos e limiares dos neurônio e volte ao passo 1.



O Perceptron

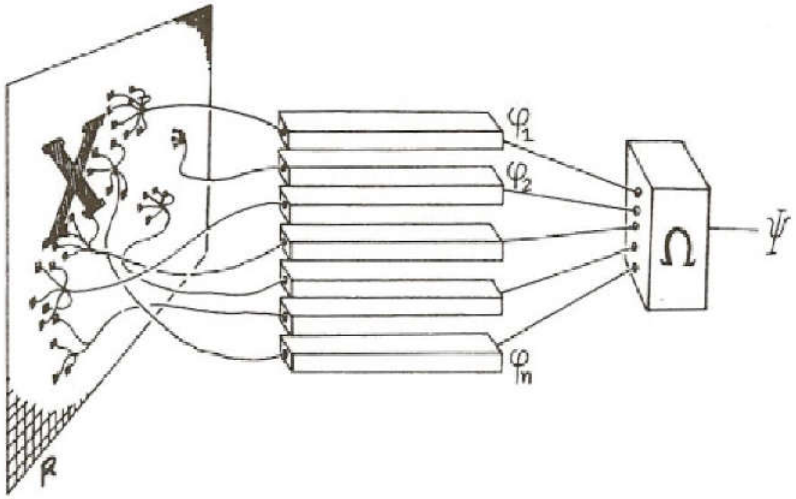
Fonte: https://imagens.canaltech.com.br/153255.273869-redes-neurais.jpg

O Perceptron: Conceitos



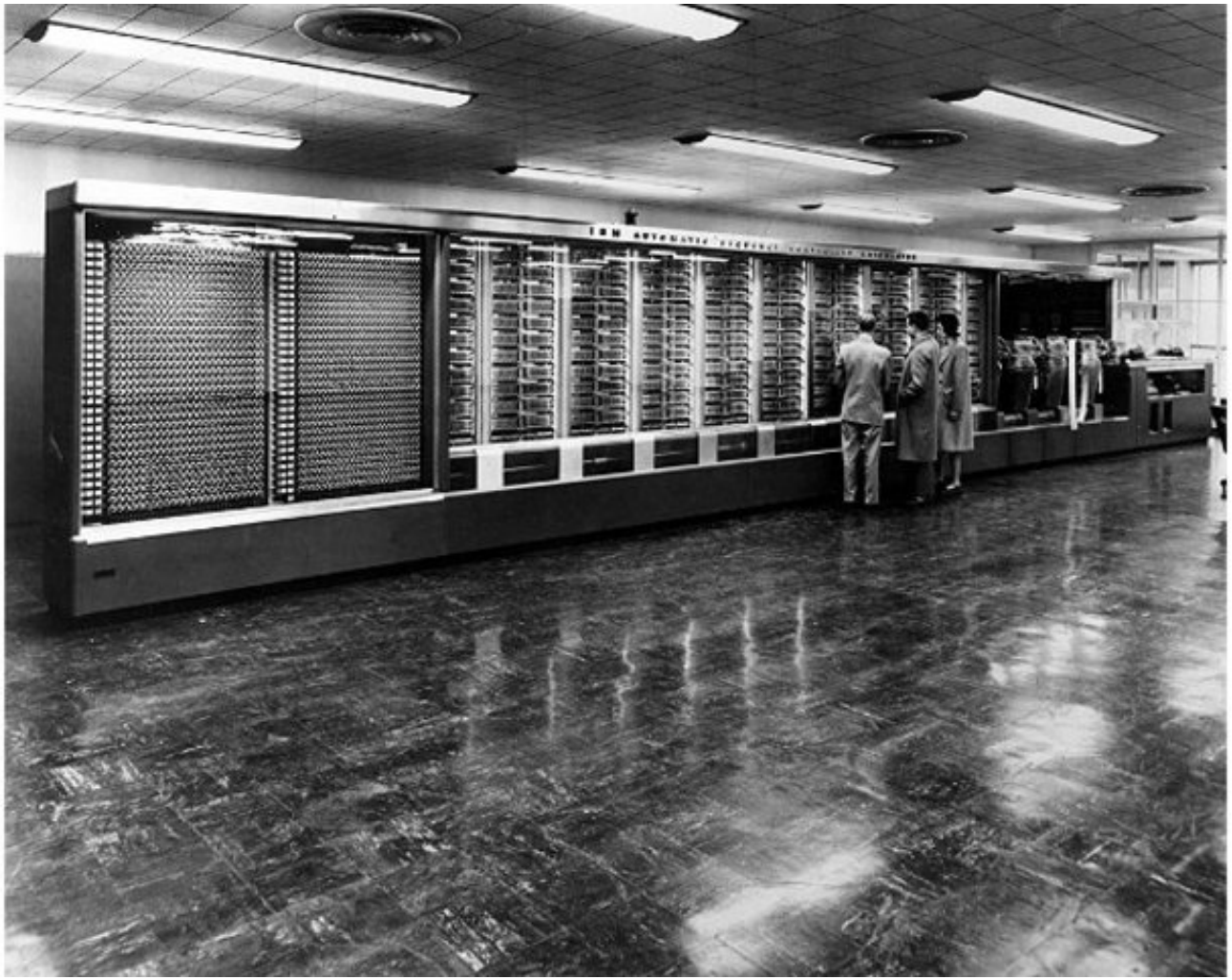
- É a forma mais simples de configuração de uma rede neural artificial (idealizada por Rosenblatt, 1958).
- Constituída de apenas uma camada, tendo-se ainda somente um neurônio nesta única camada.
- Seu propósito inicial era implementar um modelo computacional inspirado na retina, objetivando-se então um elemento de percepção eletrônica de sinais.
- Suas aplicações consistiam de identificar padrões geométricos.

Perceptron: Conceitos

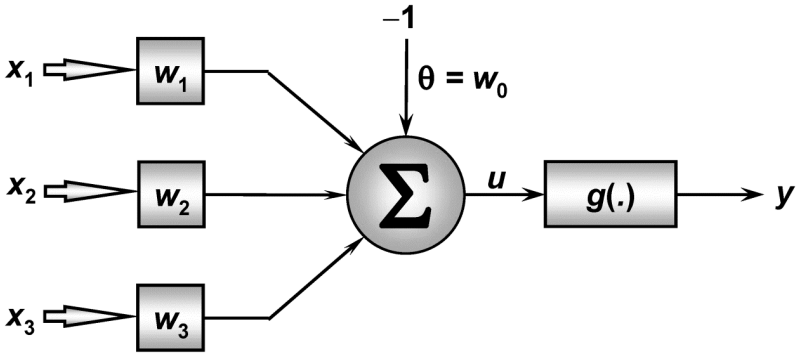


- Sinais elétricos advindos de fotocélulas mapeando padrões geométricos eram ponderados por resistores sintonizáveis.
- 2) Os resistores eram ajustados durante o processo de treinamento.
- 3) Um somador efetuava a composição de todos os sinais.
- 4) Em consequência, o Perceptron poderia reconhecer diversos padrões geométricos, tais como letras e números.

Perceptron: Conceitos



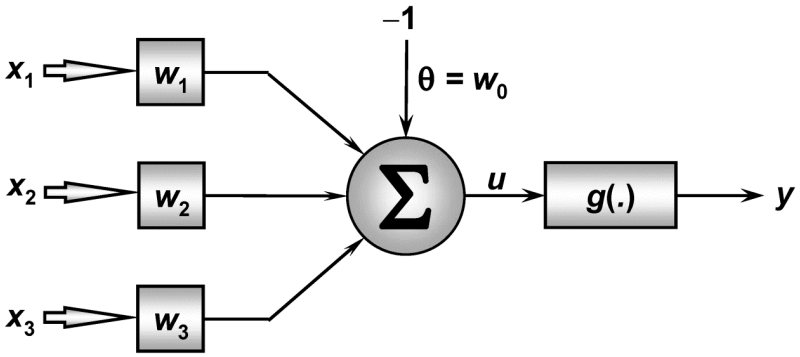
O Perceptron: Arquitetura



- **Entrada**: $x_1, x_2, ..., x_n$
- **Pesos sinápticos**: $w_1, w_2, ..., w_n$
- **Limiar de Ativação**: $\theta = w_0$
- **Função de Ativação**: $g(.)$
- **Saída**: y

- Embora seja uma rede simples, o Perceptron teve potencial de atrair, quando de sua proposição, diversos pesquisadores que aspiravam investigar essa promissora área de pesquisa.
- Recebeu ainda especial atenção da comunidade científica que também trabalhava com inteligência artificial.
- O Perceptron é tipicamente utilizado em problemas de "Classificação de Padrões".

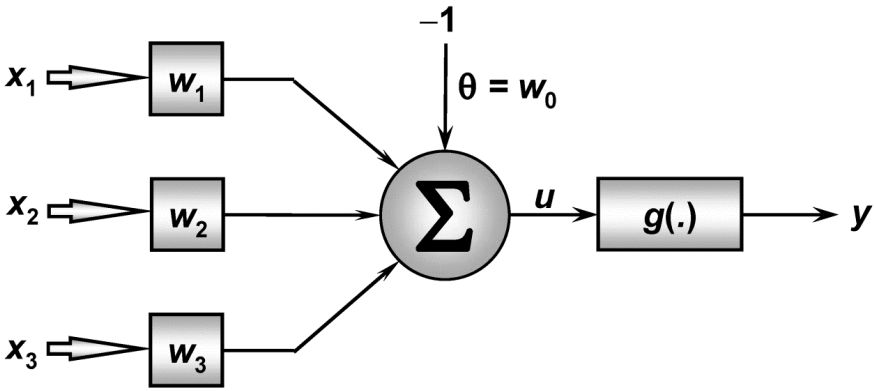
O Perceptron: Funcionamento



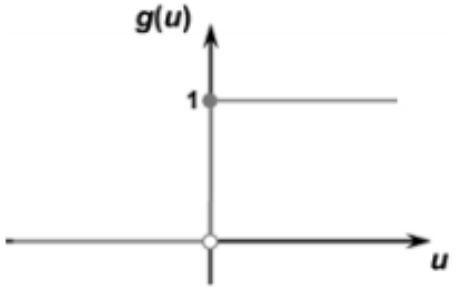
$$\begin{cases} u = \sum_{i=1}^{n} w_i \cdot x_i - \theta \\ y = g(u) \end{cases}$$

1. Apresentação de um conjunto de valores que representam as variáveis de entrada do neurônio.
2. Multiplicação de cada entrada do neurônio pelo seu respectivo peso sináptico.
3. Obtenção do potencial de ativação produzido pela soma ponderada dos sinais de entrada, subtraindo-se o limiar de ativação.
4. Aplicação de uma função de ativação apropriada, tendo-se como objetivo limitar a saída do neurônio.
5. Compilação da saída a partir da aplicação da função de ativação neural em relação ao seu potencial de ativação

Perceptron: Funções de Ativação

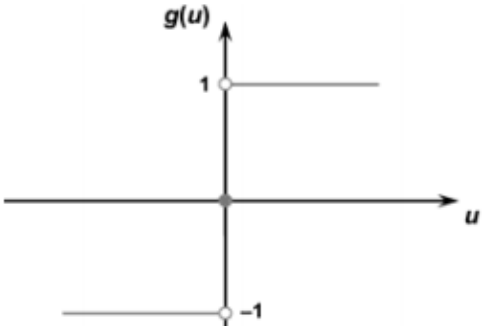


1. Função Degrau:



$$g(u) = \begin{cases} 0, & u < 0 \\ 1, & u \geq 0 \end{cases}$$

2. Função Degrau Bipolar

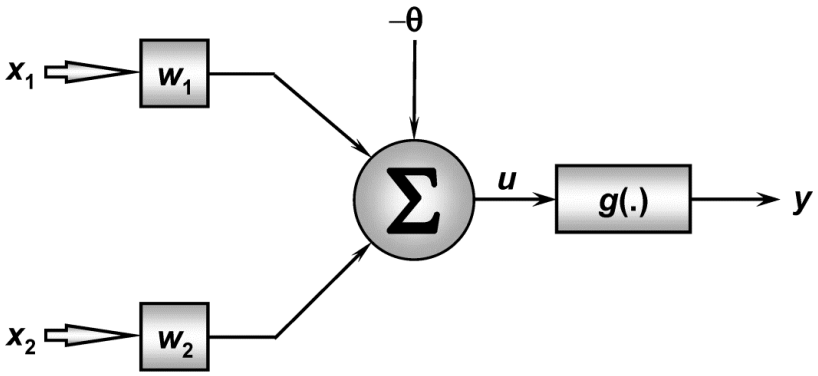


$$g(u) = \begin{cases} -1, & u < 0 \\ 1, & u \geq 0 \end{cases}$$

O Perceptron: Treinamento

Parâmetro	Variável Representativa	Tipo Característico
Entradas	x_i (i-ésima entrada)	Real ou Binário (externo)
Pesos Sinápticos	w_i (i-ésima entrada)	Real (inicializado aleatoriamente)
Limiar	θ	Real (inicializado aleatoriamente)
Saída	Y	Binária
Função de Ativação	g(.)	Degrau ou Sinal
Processo de Treinamento	---	Supervisionado
Regra de Aprendizado	---	Regra de Hebb

O Perceptron: Treinamento



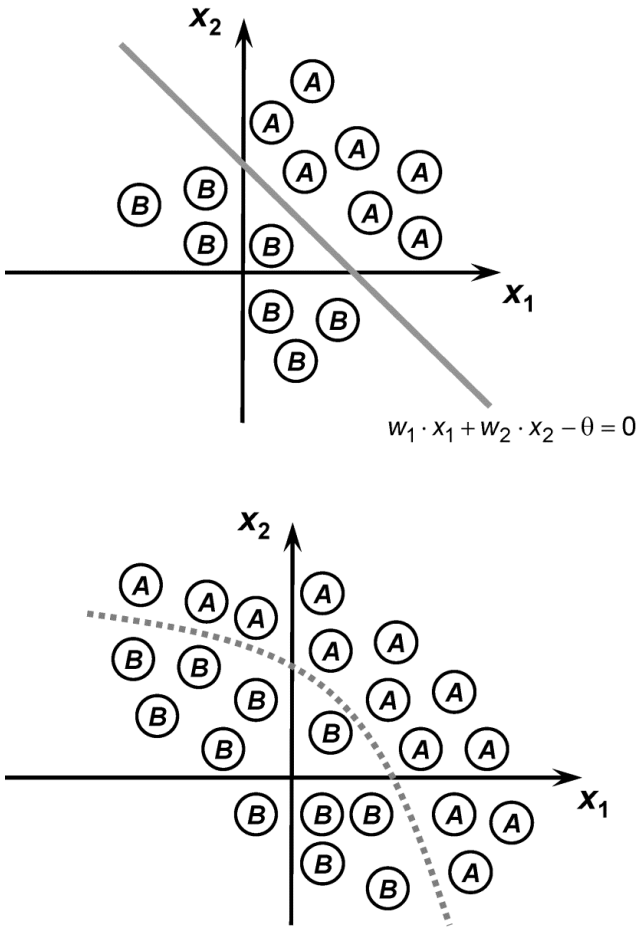
$$\begin{cases} u = \sum_{i=1}^{n} w_i \cdot x_i - \theta = w_1 . x_1 + w_2 . x_2 - \theta \\ y = g(u) \end{cases}$$

se g(.) for a função de ativação **sinal**, então:

$$y = g(u) \begin{cases} 1, se\ u = \sum_{i=1}^{n} w_i \cdot x_i - \theta \geq 0 \Rightarrow w_1 . x_1 + w_2 . x_2 - \theta \geq 0 \\ -1, se\ u = \sum_{i=1}^{n} w_i \cdot x_i - \theta < 0 \Rightarrow w_1 . x_1 + w_2 . x_2 - \theta < 0 \end{cases}$$

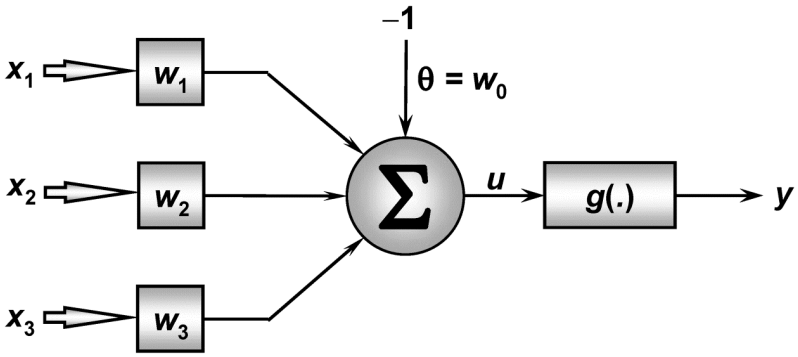
Discriminador Linear!!!

O Perceptron: Treinamento



1. Se o Perceptron fosse constituído de três entradas (três dimensões), a fronteira de separação seria representada por um plano.
2. Se o Perceptron fosse constituído de quatro ou mais entradas, suas fronteiras seriam hiperplanos.
3. **Conclusão**: a condição necessária para que o Perceptron de camada simples possa ser utilizado como um classificador de padrões é que as classes do problema a ser mapeado sejam linearmente separáveis.

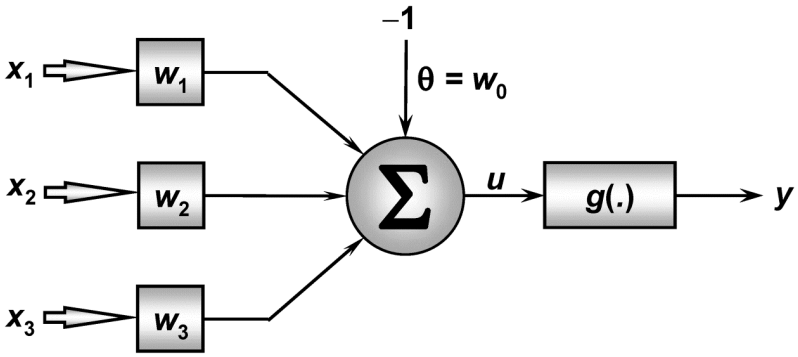
O Perceptron: Regra de Hebb



$$w = [\theta\ w_1\ w_2\ \ldots w_n]^T$$

- O processo de ajuste dos pesos e limiar do PERCEPTRON é baseado no algoritmo de aprendizado denominado de **Regra de Hebb**.
  - Se a saída produzida pelo Perceptron está coincidente com a saída desejada, os seus pesos sinápticos e limiares são então incrementados (ajuste excitatório) proporcionalmente aos valores de suas entradas.
  - Se a saída produzida pelo Perceptron é diferente do valor desejado, os pesos sinápticos e limiar serão então decrementados (ajuste inibitório).
- Este processo é repetido, sequencialmente, para todas as amostras de treinamento, até que a saída produzida pelo Perceptron seja similar à saída desejada de cada amostra.

O Perceptron: Regra de Hebb



$$w = [\theta\ w_1\ w_2\ \ldots w_n]^T$$

- a adaptação do vetor de pesos deve ser efetuada da seguinte forma:

$$w_i^{atual} = w_i^{anterior} + \eta.\left(d^{(k)} - y\right).x^{(k)}$$

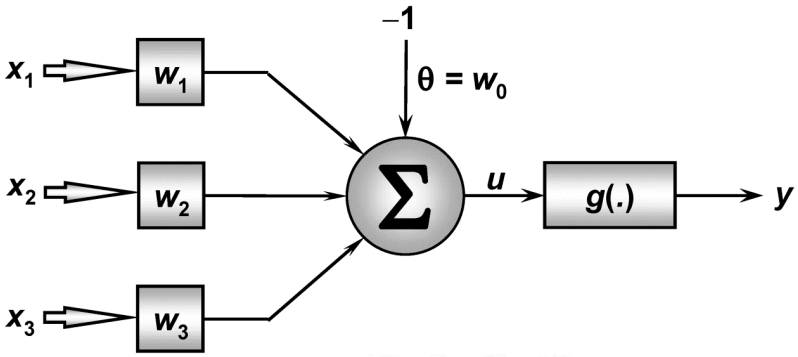
$$\theta_i^{atual} = \theta_i^{anterior} + \eta.\left(d^{(k)} - y\right).x^{(k)}$$

$$w_i^{atual} = w_i^{anterior} + \eta.\left(d^{(k)} - y\right).x^{(k)}$$

$$w = \left[-1\ x_1^{(k)}\ x_2^{(k)} x_3^{(k)}\ \ldots x_n^{(k)}\right]^T$$

Formato das amostras de treinamento

O Perceptron: Conjuntos de Treinamento



$$\Omega^{(x)} = \begin{matrix} & x^{(1)} & x^{(2)} & x^{(3)} & x^{(4)} \\ x_0 & -1 & -1 & -1 & -1 \\ x_1 & 0{,}1 & 0{,}3 & 0{,}6 & 0{,}5 \\ x_2 & 0{,}4 & 0{,}7 & 0{,}9 & 0{,}7 \\ x_3 & 0{,}7 & 0{,}2 & 0{,}8 & 0{,}1 \end{matrix}$$

$$\Omega^{(d)} = \begin{matrix} d^{(1)} & d^{(2)} & d^{(3)} & d^{(4)} \\ [\,1 & -1 & -1 & 1\,] \end{matrix}$$

- **Exemplo**: supondo um problema a ser mapeado pelo Perceptron com três entradas { x_1, x_2, x_3 }, conforme a figura ao lado.
- Assumindo-se quatro amostras da seguinte forma:
  - Amostra 1 → Entrada=[0,1 0,4 0,7] e Saída = [1]
  - Amostra 2 → Entrada=[0,3 0,7 0,2] e Saída = [-1]
  - Amostra 3 → Entrada=[0,6 0,9 0,8] e Saída = [-1]
  - Amostra 3 → Entrada=[0,5 0,7 0,1] e Saída = [1]
- Os vetores devem ficar na forma:
  - $x^{(1)} = [-1 \;\; 0{,}1 \;\; 0{,}4 \;\; 0{,}7]^T$, com $d^{(1)} = 1$
  - $x^{(2)} = [-1 \;\; 0{,}3 \;\; 0{,}7 \;\; 0{,}2]^T$, com $d^{(2)} = -1$
  - $x^{(3)} = [-1 \;\; 0{,}6 \;\; 0{,}9 \;\; 0{,}8]^T$, com $d^{(3)} = -1$
  - $x^{(4)} = [-1 \;\; 0{,}5 \;\; 0{,}7 \;\; 0{,}1]^T$, com $d^{(4)} = 1$

O Perceptron: Pseudocódigo do Algoritmo



Início {Algoritmo *Perceptron* – Fase de Treinamento}

<1> Obter o conjunto de amostras de treinamento $\{ \boldsymbol{x}^{(k)} \}$;

<2> Associar a saída desejada $\{ d^{(k)} \}$ para cada amostra obtida;

<3> Iniciar o vetor $\boldsymbol{w}$ com valores aleatórios pequenos;

<4> Especificar a taxa de aprendizagem $\{\eta\}$;

<5> Iniciar o contador de número de épocas $\{época \leftarrow 0\}$;

<6> Repetir as instruções:

- <6.1> *erro* ← "inexiste";
- <6.2> Para todas as amostras de treinamento $\{ \boldsymbol{x}^{(k)},\ d^{(k)} \}$, fazer:
  - <6.2.1> $u \leftarrow \boldsymbol{w}^T \cdot \boldsymbol{x}^{(k)}$;
  - <6.2.2> $y \leftarrow \text{sinal}(u)$;
  - <6.2.3> Se $y \neq d^{(k)}$
    - <6.2.3.1> Então $\begin{cases} \boldsymbol{w} \leftarrow \boldsymbol{w} + \eta \cdot (d^{(k)} - y) \cdot \boldsymbol{x}^{(k)} \\ erro \leftarrow \text{"existe"} \end{cases}$
- <6.3> *época* ← *época* + 1;

Até que: *erro* ← "inexiste"

Fim {Algoritmo *Perceptron* – Fase de Treinamento}

O Perceptron: Pseudocódigo do Algoritmo



Início {Algoritmo *Perceptron* – Fase de Operação}

<1> Obter uma amostra a ser classificada { $\boldsymbol{x}$ };

<2> Utilizar o vetor $\boldsymbol{w}$ ajustado durante o treinamento;

<3> Executar as seguintes instruções:

<3.1> $u \leftarrow \boldsymbol{w}^T \cdot \boldsymbol{x}$;

<3.2> $y \leftarrow \text{sinal}(u)$;

<3.3> Se $y = -1$

<3.3.1> Então: amostra $\boldsymbol{x} \in$ *{Classe A}*

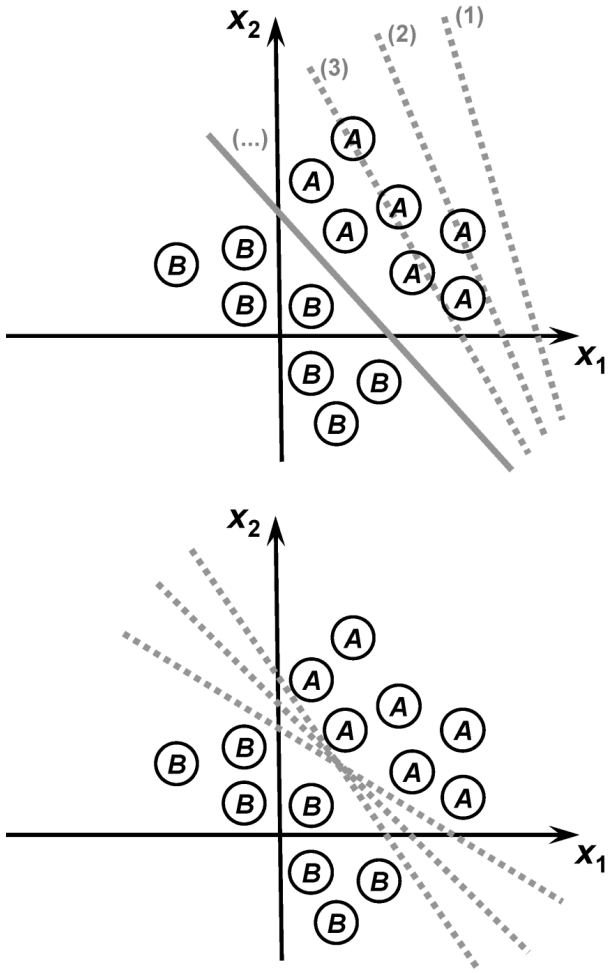
<3.4> Se $y = 1$

<3.4.1> Então: amostra $\boldsymbol{x} \in$ *{Classe B}*

Fim {Algoritmo *Perceptron* – Fase de Operação}

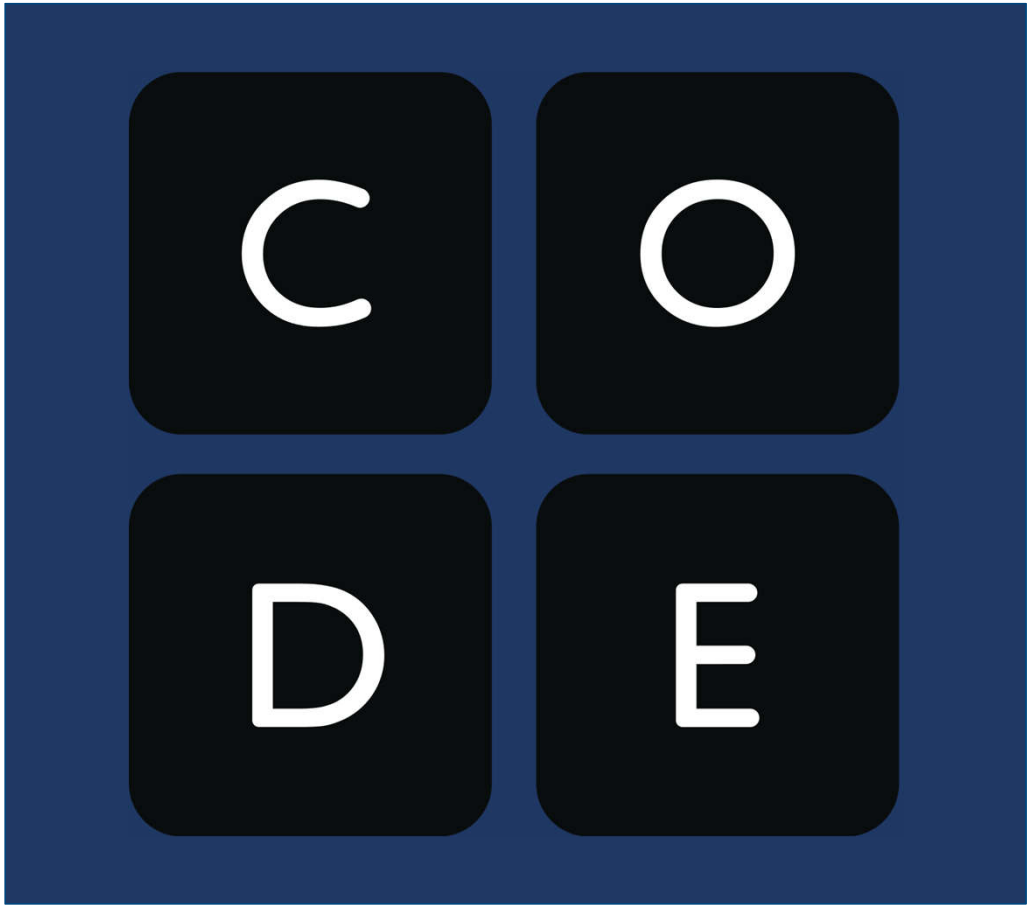
Não esqueça do -1 no vetor de entradas: $x^{(k)} = \left[-1, x_1^{(k)}, x_2^{(k)} \ldots x_n^{(k)}\right]^T$

O Perceptron: Critério de Convergência



- Processo de treinamento tende a mover continuamente o hiperplano de classificação até que seja alcançada uma fronteira de separação que permite dividir as duas classes.
- A reta de separabilidade a ser produzida após o treinamento do Perceptron não é única.
- O número de épocas pode também variar de treinamento para treinamento.

Challenge Time





Exercício 1: Questões



1. O que acontece com o algoritmo do Perceptron se o problema for não-linearmente separável?

Laboratório: Implementação de Classificador utilizando o Perceptron



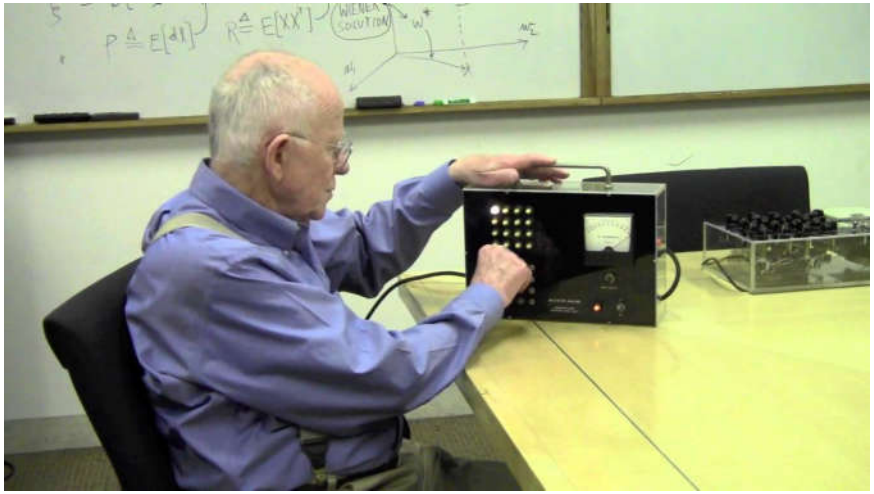
- **Utilizando Jupyter e NumPy**: faça, com o auxílio do professor os Laboratórios 1, 2 e 3.



ADALINE

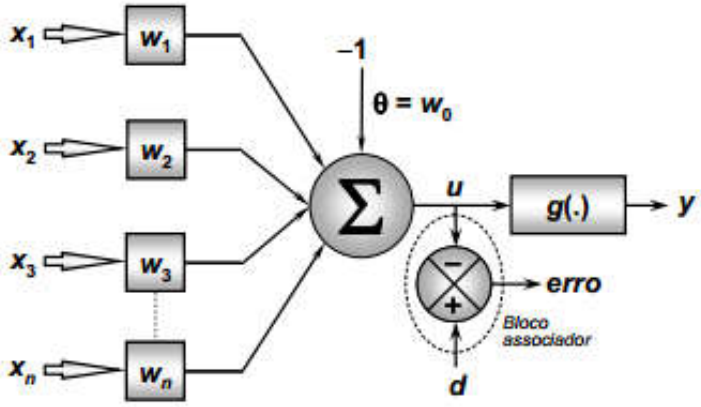
Fonte: https://imagens.canaltech.com.br/153255.273869-redes-neurais.jpg

ADALINE: Conceitos



- **Adaptive Linear Element**: idealizado por Widrow & Hoff em 1960.
- Principal aplicação: sistemas de chaveamento de circuitos telefônicos.
- Embora seja um tipo de rede bem simples (um único neurônio), o ADALINE promoveu avanços para a área de RNA:
  - Desenvolvimento do algoritmo de aprendizado **Regra Delta**.
  - Aplicações em diversos problemas práticos envolvendo processamento de sinais analógicos (primeiras aplicações industriais de redes neurais artificiais).

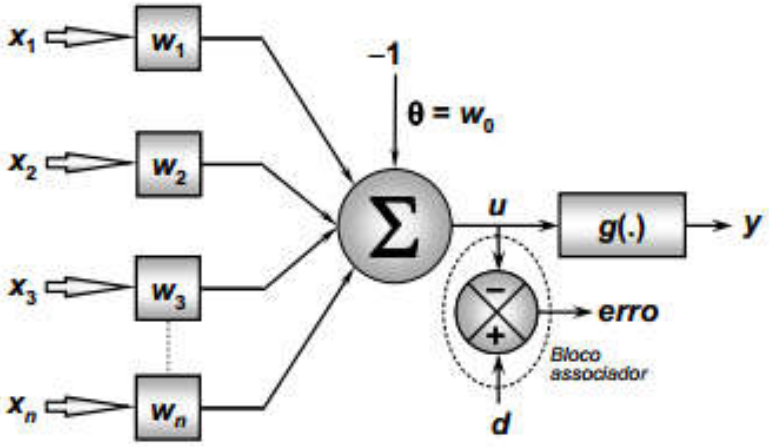
ADALINE: Arquitetura



- **Entrada**: $x_1, x_2, ..., x_n$
- **Pesos sinápticos**: $w_1, w_2, ..., w_n$
- **Limiar de Ativação**: $\theta = w_0$
- **Função de Ativação**: g(.)
- **Saída**: y
- **Valor desejado**: d
- **Erro**: diferença entre {d} e {u}

- Rede ADALINE é constituída de N sinais de entrada e somente uma saída (composta de um único neurônio).
- Assim como o Perceptron, a rede ADALINE possui também arquitetura *feedforward* de camada única, pois o fluxo de informações é realizado sempre adiante.
- Assim como o Perceptron, a rede ADALINE, devido à sua simplicidade estrutural, tem sido mais utilizada em problemas de "Classificação de Padrões", envolvendo apenas duas classes distintas.

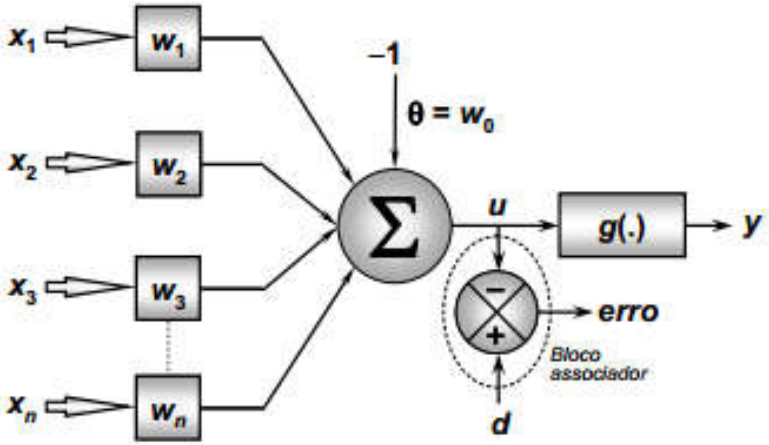
ADALINE: Funcionamento



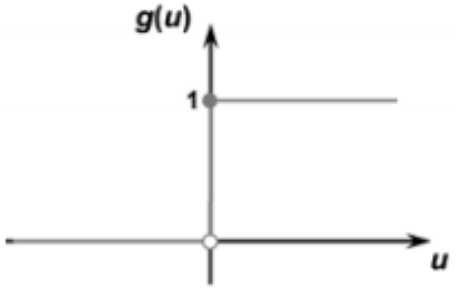
$$\begin{cases} u = \sum_{i=1}^{n} w_i \cdot x_i - \theta \\ y = g(u) \\ e = d - u \end{cases}$$

1. Apresentação de um conjunto de valores que representam as variáveis de entrada do neurônio.
2. Multiplicação de cada entrada pelo seu respectivo peso sináptico.
3. Obtenção do potencial de ativação produzido pela soma ponderada dos sinais de entrada, subtraindo-se o limiar de ativação.
4. Aplicação de uma função de ativação apropriada, tendo-se como objetivo limitar a saída do neurônio.
5. Compilação da saída a partir da aplicação da função de ativação neural em relação ao seu potencial de ativação.
6. **Presença de bloco associador, cuja função é produzir o sinal de "erro" entre "d" e "u" a fim de auxiliar no treinamento da rede**.

ADALINE: Funções de Ativação

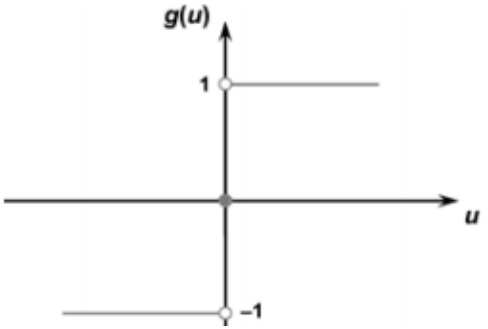


1. Função Degrau:



$$g(u) = \begin{cases} 0, & u < 0 \\ 1, & u \geq 0 \end{cases}$$

2. Função Degrau Bipolar

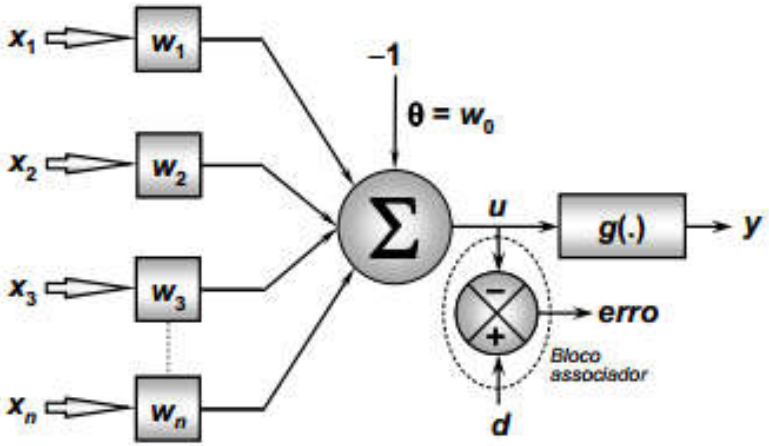


$$g(u) = \begin{cases} -1, & u < 0 \\ 1, & u \geq 0 \end{cases}$$

ADALINE: Treinamento

Parâmetro	Variável Representativa	Tipo Característico
Entradas	x_i (i-ésima entrada)	Real ou Binário (externo)
Pesos Sinápticos	w_i (i-ésima entrada)	Real (inicializado aleatoriamente)
Limiar	θ	Real (inicializado aleatoriamente)
Saída	Y	Binária
Função de Ativação	g(.)	Degrau ou Sinal
Processo de Treinamento	---	Supervisionado
Regra de Aprendizado	---	**Regra Delta**

ADALINE: Treinamento



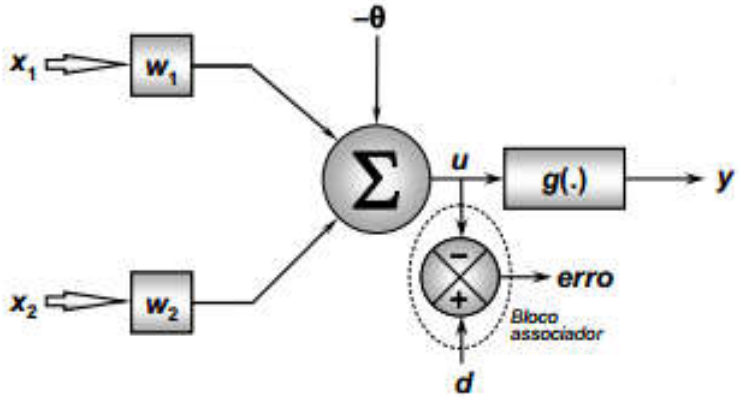
1. Assim como no *Perceptron*, o ajuste dos pesos e limiar do ADALINE é efetuado utilizando processo de **Treinamento Supervisionado**.

2. Então, para cada amostra dos sinais de entrada se tem a respectiva saída (resposta) desejada.

3. Como o **ADALINE** é tipicamente usado em problemas de classificação de padrões, a sua saída pode assumir somente dois valores possíveis, os quais se associam às “duas classes” que o mesmo irá identificar.

4. A diferença principal entre o **ADALINE** e o *Perceptron* está principalmente na regra de aprendizado utilizada para os ajustes dos pesos e limiar.

ADALINE: Treinamento



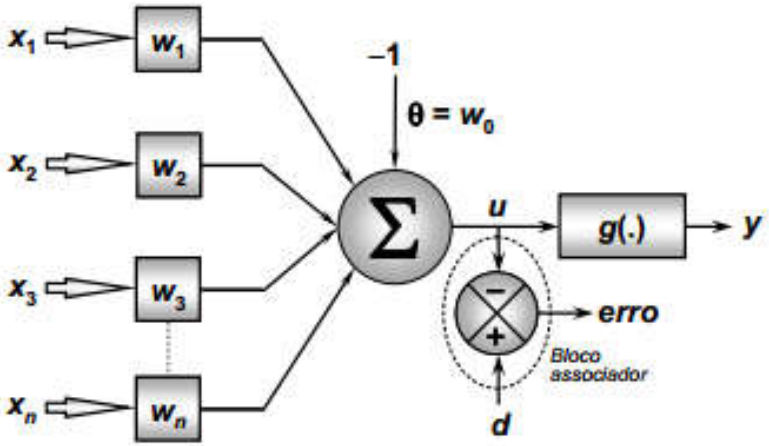
$$\begin{cases} u = \sum_{i=1}^{n} w_i \cdot x_i - \theta = w_1 . x_1 + w_2 . x_2 - \theta \\ y = g(u) \end{cases}$$

se g(.) for a função de ativação **sinal**, então:

$$y = g(u) \begin{cases} 1, se\ u = \sum_{i=1}^{n} w_i \cdot x_i - \theta \geq 0 \Rightarrow w_1 . x_1 + w_2 . x_2 - \theta \geq 0 \\ -1, se\ u = \sum_{i=1}^{n} w_i \cdot x_i - \theta < 0 \Rightarrow w_1 . x_1 + w_2 . x_2 - \theta < 0 \end{cases}$$

Discriminador Linear!!!

ADALINE: Regra Delta

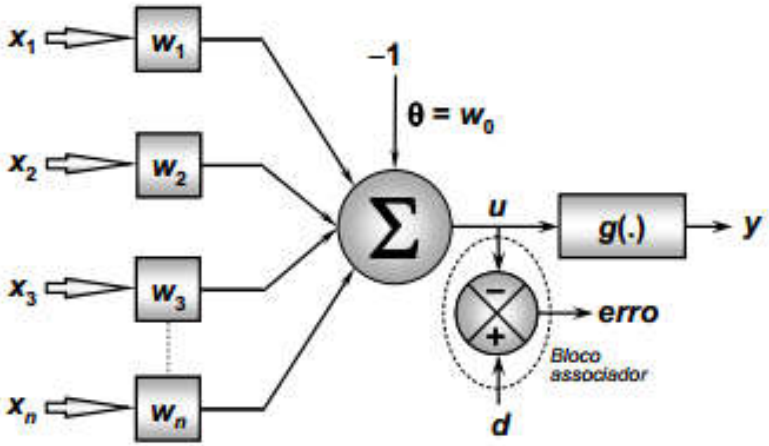


$$w = [\theta\ w_1\ w_2\ \ldots w_n]^T$$

- O processo de ajuste dos pesos e limiar do ADALINE é baseado no algoritmo de aprendizado denominado de **Regra Delta**, também conhecida como Algoritmo LMS (***Least Mean Square***) ou **Método do Gradiente Descendente**.

- Assumindo-se a disponibilidade de p amostras de treinamento, o princípio envolvido com a aplicação da Regra Delta, objetivando ajustar os pesos e limiar do neurônio, é a seguinte:
  - Minimizar a diferença (erro) entre a saída desejada **d** e a resposta do combinador **u**, considerando-se para tanto todas as **p** amostras.
  - Mais especificamente, utiliza-se a minimização do **Erro Quadrático** entre **u** e **d** com o intuito de ajustar o vetor de pesos {**w**} da rede.

ADALINE: Regra Delta



$$w = [\theta\ w_1\ w_2\ \ldots w_n]^T$$

- Consiste de obter um w* ótimo tal que o **Erro Quadrático** $\{E(w^*)\}$ sobre todo o conjunto de amostras seja o mínimo possível, isto é:

$$E(w^*) \leq E(w), \forall w \in R^{n+1}$$

- A função **Erro Quadrático** em relação às p amostras de treinamento é definida por:

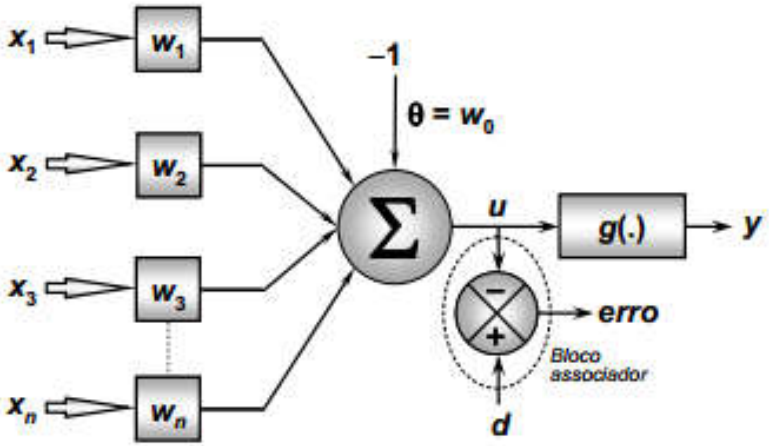
$$E(w) = \frac{1}{2}\sum_{k=1}^{p}\left(d^k - u\right)^2 \Rightarrow E(w) = \frac{1}{2}\sum_{k=1}^{p}\left(d^k - \left(\sum_{i=1}^{n} w_i \cdot x_i - \theta\right)\right)^2$$

- Em forma vetorial, temos:

$$E(w) = \frac{1}{2}\sum_{k=1}^{p}\left(d^k - \left(w^T.x^k - \theta\right)\right)^2$$

- Esta expressão totaliza o erro quadrático contabilizando-se as **p** amostras de treinamento.

ADALINE: Regra Delta



$$w = [\theta\ w_1\ w_2\ \ldots w_n]^T$$

- **Gradiente Descendente**: Consiste de aplicar o operador gradiente em relação ao vetor w, tendo-se como objetivo a busca do valor ótimo para o erro quadrático, isto é:
- A função Erro Quadrático em relação às p amostras de treinamento é definida por:

$$\nabla E(w) = \frac{\partial E(w)}{\partial w}, \qquad \text{onde } E(w) = \frac{1}{2}\sum_{k=1}^{p}\left(d^k - \left(w^T \cdot x^k - \theta\right)\right)^2$$

- Substituindo E(w) na expressão do gradiente:

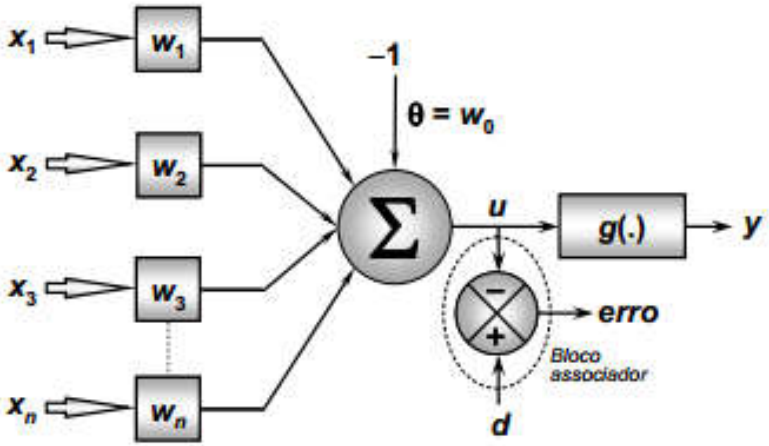
$$\nabla E(w) = \sum_{k=1}^{p}\left(d^k - \left(w^T.x^k - \theta\right)\right).(-x^{(k)})$$

- Retornando o valor de u na expressão acima:

$$\nabla E(w) = -\sum_{k=1}^{p}\left(d^k - u\right).(x^{(k)})$$

Gradiente do Erro em relação as amostras de treinamento

ADALINE: Regra Delta



$$w = [\theta\ w_1\ w_2\ \ldots w_n]^T$$

- a adaptação do vetor de pesos deve ser efetuada na direção oposta aquela do gradiente, pois o objetivo da otimização é minimizar o **erro quadrático médio** entre **d** e **u**.
- Nesta condição, a variação Δw a ser aplicada ao vetor de pesos do ADALINE é dada por:

$$\Delta w = -\eta . \nabla E(w)$$

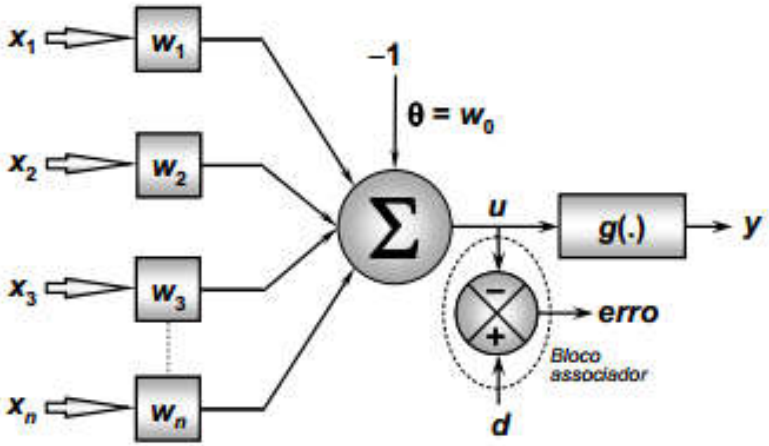
- Substituindo $\nabla E(w)$ na expressão de ajuste:

$$\Delta w = -\eta . \sum_{k=1}^{p} \left(d^k - u\right) . \left(x^{(k)}\right)$$

- A correção de pesos é dada por:

$$w^{atual} = w^{anterior} + \Delta w$$

ADALINE: Regra Delta – Ajuste de algoritmo



- Por razões de simplificação computacional, a atualização de **w** pode ser também realizada de forma discreta, onde um passo de ajuste em **w** é realizado após a apresentação de cada k-ésima amostra de treinamento:

$$w^{atual} = w^{anterior} + \eta . \sum_{k=1}^{p} \left(d^k - u\right) . \left(x^{(k)}\right)$$

$$w^{atual} = w^{anterior} + \eta . \left(d^{(k)} - u\right) . x^{(k)}$$

Onde:

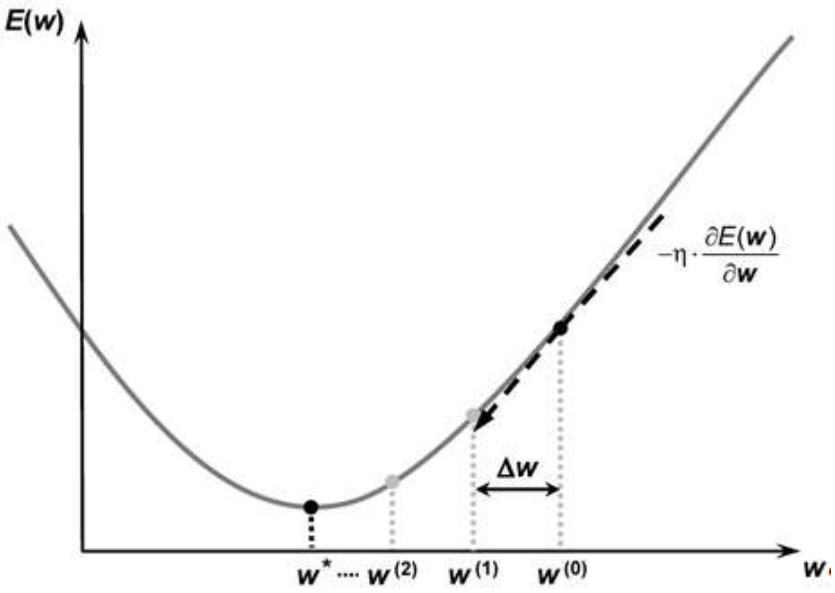
$w = [\theta, w_1, w_2, \ldots w_n]^T$

η é a taxa de aprendizagem {0 < η< 1}.

$x^{(k)} = \left[-1, x_1^{(k)}, x_2^{(k)} \ldots x_n^{(k)}\right]^T$

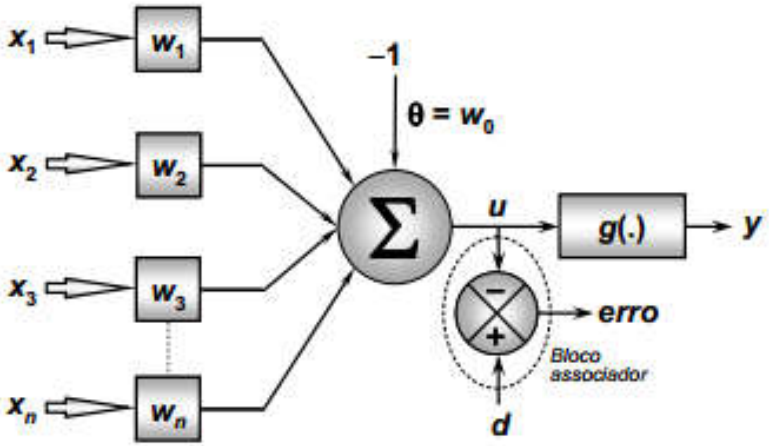
$d^{(k)}$ é o valor desejado para k-ésima amostra de treinamento.

ADALINE: O Gradiente Descendente



- A interpretação geométrica frente aos passos de atualização de **w** rumo ao ponto de minimização **w*** da função erro quadrático $\{E(w)\}$, é dada pela figura ao lado:
  1. Parte-se de um valor inicial $w^{(0)}$;
  2. O próximo valor de **w** (dado por $w^{(1)}$) será obtido considerando-se a direção oposta ao vetor gradiente em relação a $w^{(0)}$;
  3. Para o próximo passo de atualização, o ajuste de **w** (representado agora por $w^{(2)}$) será realizado considerando-se o valor do gradiente em relação à $w^{(1)}$;
  4. Aplicando-se tais passos sucessivamente, haverá a convergência em direção a **w***;
  5. Este valor de **w*** será a configuração ótima para os parâmetros livres do ADALINE;
  6. O valor de $E(w*)$ será então sempre menor que quaisquer $E(w)$ calculados nos passos anteriores.

ADALINE: Regra Delta – Critério de parada do algoritmo



- Com a aplicação dos passos iterativos anteriores, o vetor **w** caminhará em direção ao seu valor ótimo $\{w*\}$.
- Para se detectar o alcance de $w*$, a fim de cessar o processo de convergência, há a necessidade de se estabelecer um critério de parada.
- Para tanto, utiliza-se o valor do erro quadrático médio $\{E_{qm}(w)\}$ em relação a todas as amostras de treinamento, sendo este definido por:

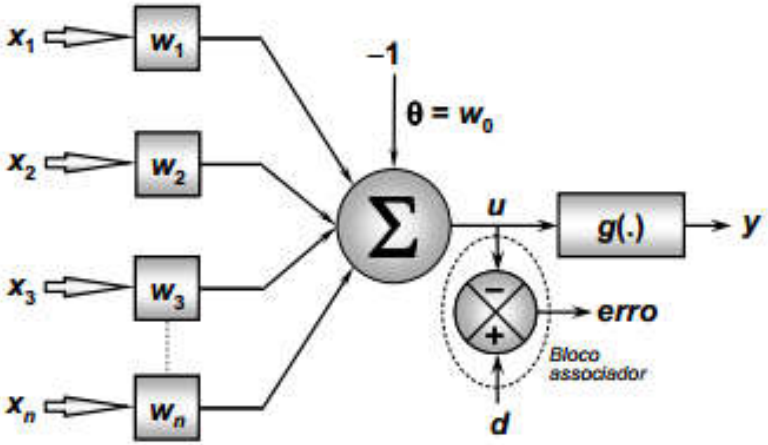
$$E_{qm}(w) = \frac{1}{p}\sum_{k=1}^{p}\left(d^{(k)} - u\right)^2$$

- **Critério de Parada:**

$$\left|E_{qm}\left(w^{atual}\right) - E_{qm}\left(w^{anterior}\right)\right| \leq \varepsilon$$

onde ε é a precisão requerida para o processo de convergência

ADALINE: Conjuntos de Treinamento



$$\Omega^{(x)} = \begin{bmatrix} & x^{(1)} & x^{(2)} & x^{(3)} & x^{(4)} \\ x_0 & -1 & -1 & -1 & -1 \\ x_1 & 0{,}1 & 0{,}3 & 0{,}6 & 0{,}5 \\ x_2 & 0{,}4 & 0{,}7 & 0{,}9 & 0{,}7 \\ x_3 & 0{,}7 & 0{,}2 & 0{,}8 & 0{,}1 \end{bmatrix}$$

$$\Omega^{(d)} = \begin{bmatrix} d^{(1)} & d^{(2)} & d^{(3)} & d^{(4)} \\ 1 & -1 & -1 & 1 \end{bmatrix}$$

- Exemplo: supondo um problema a ser mapeado pelo **ADALINE** com três entradas { x1, x2, x3 }, conforme a figura ao lado.
- Assumindo-se quatro amostras da seguinte forma:
  - Amostra 1 → Entrada=[0,1 0,4 0,7] e Saída = [1]
  - Amostra 2 → Entrada=[0,3 0,7 0,2] e Saída = [-1]
  - Amostra 3 → Entrada=[0,6 0,9 0,8] e Saída = [-1]
  - Amostra 3 → Entrada=[0,5 0,7 0,1] e Saída = [1]
- Os vetores devem ficar na forma:
  - $x^{(1)} = [-1 \quad 0{,}1 \quad 0{,}4 \quad 0{,}7]^T$, com $d^{(1)} = 1$
  - $x^{(2)} = [-1 \quad 0{,}3 \quad 0{,}7 \quad 0{,}2]^T$, com $d^{(2)} = -1$
  - $x^{(3)} = [-1 \quad 0{,}6 \quad 0{,}9 \quad 0{,}8]^T$, com $d^{(3)} = -1$
  - $x^{(4)} = [-1 \quad 0{,}5 \quad 0{,}7 \quad 0{,}1]^T$, com $d^{(4)} = 1$

ADALINE: Pseudocódigo do Algoritmo



Início {Algoritmo *Adaline* – Fase de Treinamento}

<1> Obter o conjunto de amostras de treinamento $\{ \boldsymbol{x}^{(k)} \}$;

<2> Associar a saída desejada $\{ d^{(k)} \}$ para cada amostra obtida;

<3> Iniciar o vetor $\boldsymbol{w}$ com valores aleatórios pequenos;

<4> Especificar taxa de aprendizagem $\{ \eta \}$ e precisão requerida $\{ \varepsilon \}$;

<5> Iniciar o contador de número de épocas { *época* $\leftarrow 0$ };

<6> Repetir as instruções:

- <6.1> $E_{qm}^{anterior} \leftarrow E_{qm}(\boldsymbol{w})$;
- <6.2> Para todas as amostras de treinamento $\{ \boldsymbol{x}^{(k)}, d^{(k)} \}$, fazer:
  - <6.2.1> $u \leftarrow \boldsymbol{w}^T \cdot \boldsymbol{x}^{(k)}$;
  - <6.2.2> $\boldsymbol{w} \leftarrow \boldsymbol{w} + \eta \cdot (d^{(k)} - u) \cdot \boldsymbol{x}^{(k)}$;
- <6.3> *época* $\leftarrow$ *época* + 1;
- <6.4> $E_{qm}^{atual} \leftarrow E_{qm}(\boldsymbol{w})$;

Até que: $\left| E_{qm}^{atual} - E_{qm}^{anterior} \right| \leq \varepsilon$

Fim {Algoritmo *Adaline* – Fase de Treinamento}

ADALINE: Pseudocódigo do Algoritmo



Início {Algoritmo *Adaline* – Fase de Operação}

<1> Obter uma amostra a ser classificada { $\boldsymbol{x}$ };
<2> Utilizar o vetor $\boldsymbol{w}$ ajustado durante o treinamento;
<3> Executar as seguintes instruções:
  <3.1> $u \leftarrow \boldsymbol{w}^T \cdot \boldsymbol{x}$;
  <3.2> $y \leftarrow \text{sinal}(u)$;
  <3.3> Se $y = -1$
    <3.3.1> Então: amostra $\boldsymbol{x} \in$ *{Classe A}*
  <3.4> Se $y = 1$
    <3.4.1> Então: amostra $\boldsymbol{x} \in$ *{Classe B}*

Fim {Algoritmo *Adaline* – Fase de Operação}

Não esqueça do -1 no vetor de entradas: $x^{(k)} = \left[-1, x_1^{(k)}, x_2^{(k)} \ldots x_n^{(k)}\right]^T$

ADALINE: Pseudocódigo do Algoritmo E_{QM}



Início {Algoritmo EQM}

<1> Obter a quantidade de padrões de treinamento { p };

<2> Iniciar a variável E_{qm} com valor zero { $E_{qm} \leftarrow 0$ };

<3> Para todas as amostras de treinamento { $\boldsymbol{x}^{(k)}$, $d^{(k)}$ }, fazer:

<3.1> $u \leftarrow \boldsymbol{w}^T \cdot \boldsymbol{x}^{(k)}$;

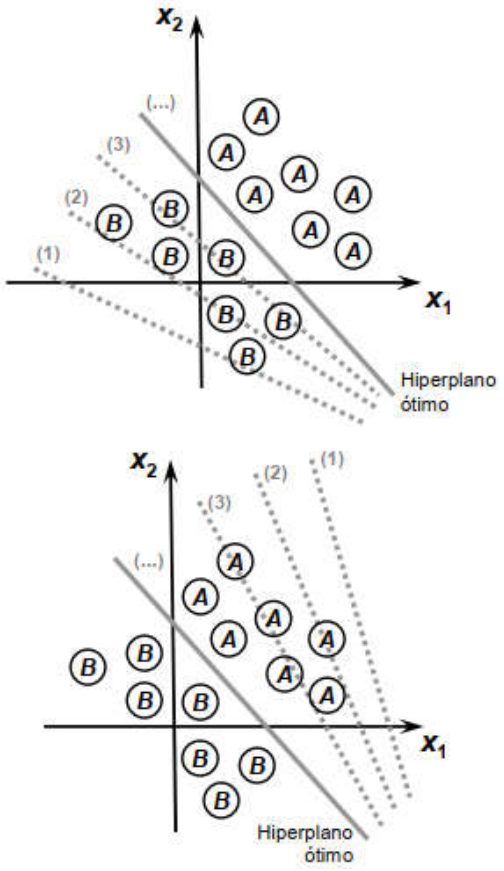
<3.2> $E_{qm} \leftarrow E_{qm} + (d^{(k)} - u)^2$;

<4> $E_{qm} \leftarrow \dfrac{E_{qm}}{p}$;

Fim {Algoritmo EQM}

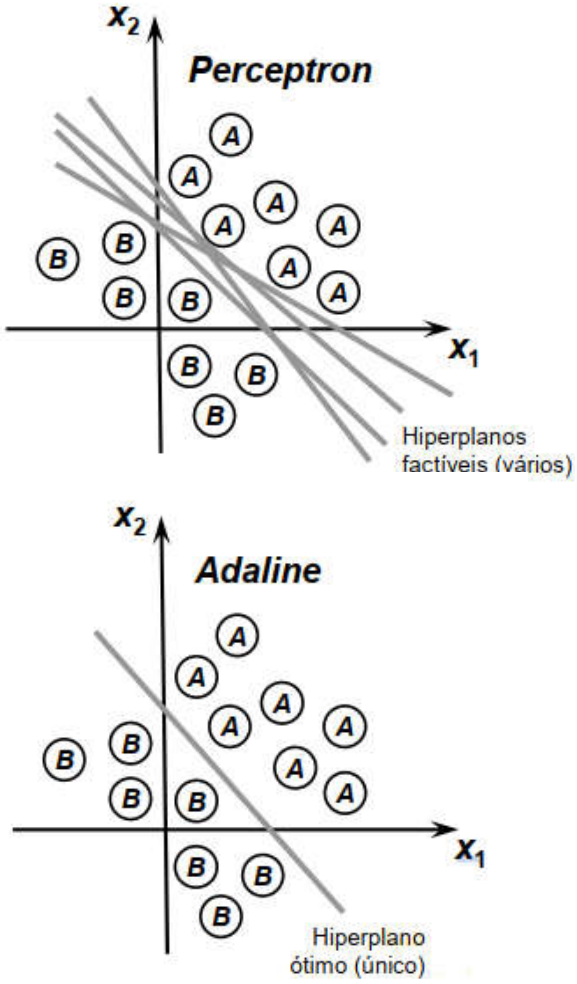
1. O algoritmo do EQM deve ser implementado como sub-rotina (função).
2. Todas as amostras de treinamento devem ser contabilizadas no cálculo do valor do EQM.

ADALINE: Critério de Convergência



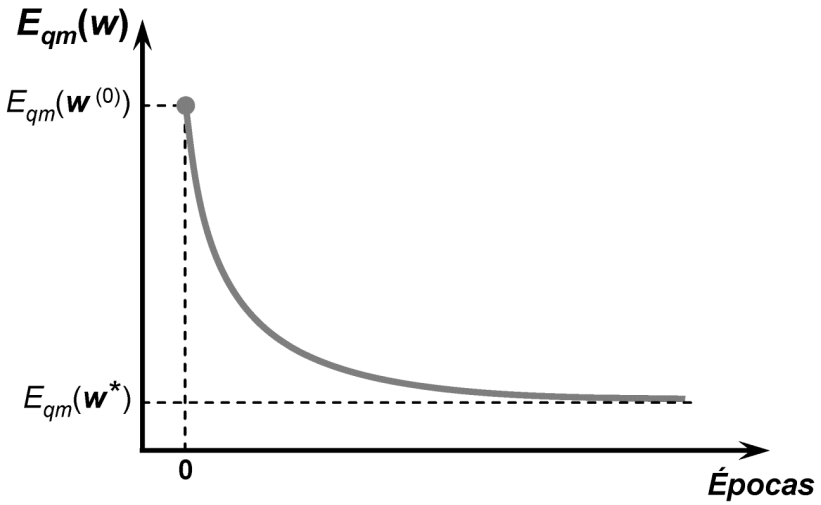
- O processo de treinamento do ADALINE tende a mover continuamente o seu vetor de pesos, com o intuito de minimizar o erro quadrático em relação a todas as amostras disponíveis para o aprendizado.
- As figuras mostram duas situações que ilustram a convergência do ADALINE rumo à estabilização, considerando-se para fins didáticos apenas duas entradas $\{x_1 \, e \, x_2\}$;
- Observa-se que nestas duas situações, cujos vetores de pesos iniciais $w^{(0)}$ foram iniciados em posições distintas, o processo de convergência encaminha o hiperplano sempre para o mesmo local.
- Nesses casos, o valor correspondente a **w*** é sempre aquele que minimiza a função erro quadrático.

ADALINE x Perceptron



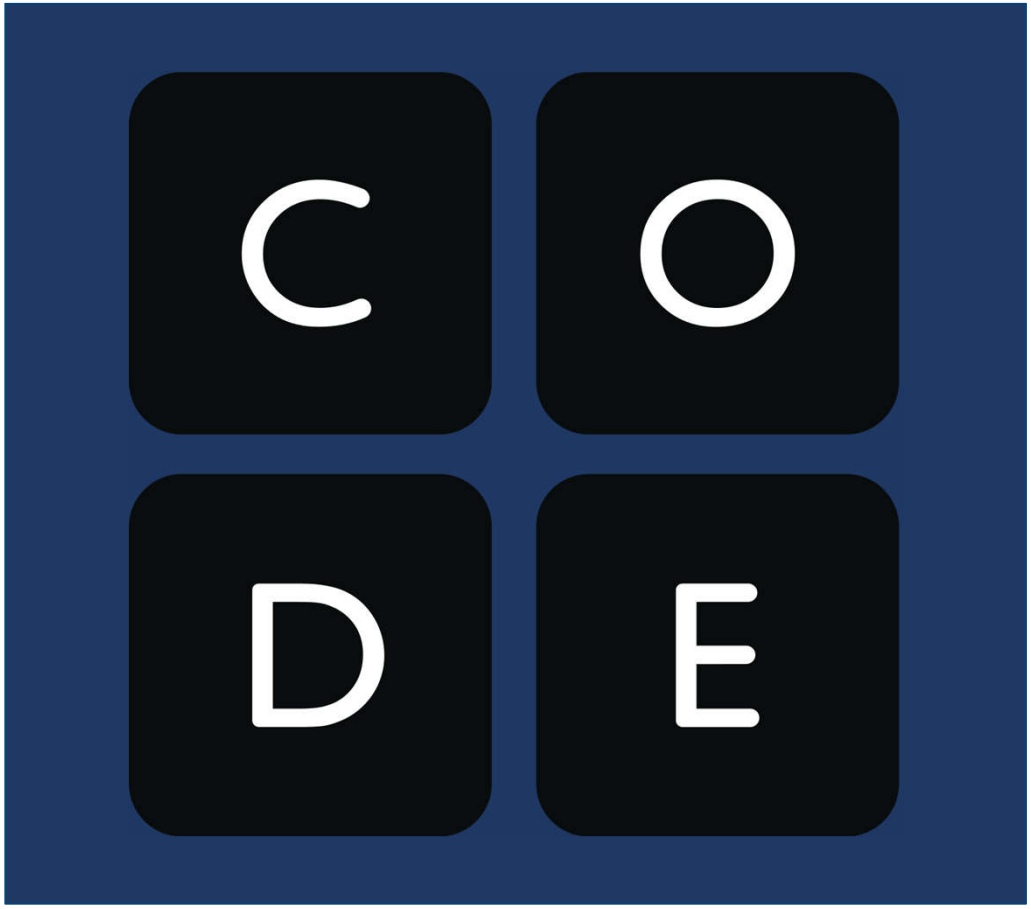
- **Perceptron**: Treinamento usando **Regra de Hebb** (Avaliação das respostas produzidas após apresentação de cada amostra de treinamento).
  - Quaisquer hiperplanos posicionados dentro da faixa de separabilidade entre as classes são considerados soluções factíveis.
- **ADALINE**: Treinamento usando **Regra Delta** (Minimização do erro em relação a todas as amostras de treinamento).
  - Independentemente dos valores iniciais atribuídos a w, o hiperplano de separabilidade (após convergência) sempre será o mesmo.

ADALINE: Comportamento da função E_{QM}



- Para fins de interpretação, a figura mostra o comportamento do erro quadrático médio $\{E_{qm}\}$ em função do número de épocas de treinamento.
- Esta decresce na medida em que as épocas de treinamento vão sendo executadas.
- Constata-se então que a curva do erro quadrático médio para o ADALINE é sempre descendente.
- Finalmente, estabiliza-se num valor constante quando o ponto de mínimo da função erro quadrático médio for alcançado

Challenge Time





Exercício 1: Questões



1. Considerando-se que um problema a ser mapeado pelo ADALINE não seja linearmente separável, explique então se para esta situação o processo de treinamento (por meio do algoritmo regra Delta) também convergirá.

2. Explique por que o treinamento do ADALINE se processa normalmente de forma mais rápido que aquele do Perceptron.

Laboratório: Implementação de Classificador utilizando o ADALINE



- **Utilizando Jupyter e NumPy**: faça, com o auxílio do professor os Laboratórios 1, 2 e 3.

Referências



- https://www.coursera.org/
- https://python.org.br/
- http://scikit-learn.org/stable/
- https://www.w3schools.com/python
- https://www.gnu.org/software/octave/
- https://octave-online.net/
- https://www.mathworks.com/products/matlab.html

Referências



- RUSSELL, STUART J.. Inteligência Artificial. 2ª ed. São Paulo: Campus - Elsevier, 2004.
- FERNANDES, Anita M. R.. Inteligência Artificial: noções gerais. 1ª ed. Florianópolis: Visual Books, 2008. Capítulo 1.
- SILVA, I.N. – Sistemas Inteligentes – Notas de Aula da disciplina de Sistemas Inteligentes – USP São Carlos, 2012.
- SILVA, I.N., SPATTI, D.H., FLAUZINO, R.A., Redes Neurais Artificiais para Engenharia e Ciências Aplicadas. 1ª ed. São Paulo: Editora Artiliber, 2010.
- NASCIMENTO JUNIOR, Cairo L., YONEYAMA, T. Inteligência Artificial em Controle e Automação. 1ª ed. São Paulo: Edgard Blücher, 2000.


Facens
AQUI TEM ENGENHARIA